

NATIONAL ECONOMICS UNIVERSITY
COLLEGE OF TECHNOLOGY
FACULTY OF DATA SCIENCE & ARTIFICIAL INTELLIGENCE (FDA)

OFFICIAL MIDTERM EXAMINATION
Academic Year: 2026–2027
Semester: Fall 2026 Course Code:
DSAI1003

FUNDAMENTAL PROGRAMMING CONCEPTS IN PYTHON

PROGRESS EXAM: DECISION MAKING & BOOLEAN LOGIC

Paper Code: _____ | Duration: 60 Minutes (Excluding paper distribution)

Candidate Full Name:

Student ID (MSSV): ...

Class / Cohort:

Exam Room / Seat No.:

Section	A	B	C	D	Total (100)	Examiner Signature
Max Score	20	25	25	30	100	
Score						

Instructions: Closed-book examination. All electronic devices and internet access are prohibited. Answer all questions directly in the designated spaces. In programming tasks, apply conditional statements (`if`, `elif`, `else`), relational/Boolean operators, and input validation. Do **not** use loops or custom functions (`def`) unless explicitly requested. Assume users enter data in expected types unless validation is specifically instructed. Do not detach pages.

MULTIPLE-CHOICE ANSWER SHEET (Write A, B, C, or D in the corresponding box):

Question	1	2	3	4	5	6	7	8	9	10
Answer										

Section A: Decision Making & Boolean Logic Concepts

(20 points)

Answer all 10 questions. Each correct answer carries 2 points. Suggested time: 12 minutes.

- Q1.** Which statement correctly describes compound statements and code blocks in Python?
- A colon (`:`) is optional after the `if` condition as long as the block is indented.
 - Every branch block must be consistently indented; mixing tabs and spaces raises an `IndentationError`.
 - Python uses curly braces `{}` to delimit blocks; indentation is solely for readability.
 - An `else` clause can exist independently without a preceding `if` or `elif` statement.
- Q2.** What happens when executing the statement `if x = 10:` in Python?
- It assigns `10` to `x` and executes the branch because `10` is considered truthy.
 - It checks if `x` equals `10` and returns a boolean value.
 - It causes a `SyntaxError` because `=` is assignment, not the equality operator `==`.
 - It automatically converts `x` to an integer before comparing.
- Q3.** Why is the direct comparison `if x == 0.3:` hazardous when `x = 0.1 + 0.2`, and what is the standard fix?
- Relational operators cannot compare floats; one must convert to strings using `str(x) == "0.3"`.
 - Binary representation causes roundoff error; one should verify `abs(x - 0.3) < 1E-14`.
 - Floating-point values are automatically rounded to integers inside conditional statements.
 - One must always use the identity operator instead: `if x is 0.3:`.
- Q4.** In Python, what is the boolean value of the expression `"apple" < "Banana"` and what is the reason?
- True, because "apple" precedes "Banana" in standard alphabetical order.
 - False, because uppercase letters have smaller Unicode values than lowercase letters ('B' is 66, 'a' is 97).
 - True, because "apple" has fewer characters than "Banana".
 - A `TypeError` is raised because strings cannot be compared with relational operators.
- Q5.** Given `score = 85` in `if score >= 60: g = "D"` followed by `elif score >= 80: g = "B"` and `elif score >= 90: g = "A"`, what is `g`?
- "B", because `85 ≥ 80` is the most specific matching range.
 - "D", because `score >= 60` is evaluated first and evaluates to True; remaining branches are skipped.
 - "A", because Python evaluates `elif` branches in descending numerical order.
 - "F", because matching multiple conditions produces an ambiguous branch error.

- Q6.** Following Python's operator precedence rules, what is the truth value of `not False and False or True`?
- True, because `not` evaluates first (True), then `and` (False), then `or` (True).
 - False, because `and` has higher precedence than `not`.
 - False, because boolean operators are evaluated strictly from left to right.
 - A `TypeError` occurs because parentheses are required when mixing logical operators.
- Q7.** Given `x = 0`, what is the outcome of evaluating the expression `x != 0 and (10 / x > 2)`?
- A `ZeroDivisionError` is raised because `10 / x` is evaluated.
 - False, because the first operand is False, so Python short-circuits and skips the division.
 - True, because Python treats division by zero as positive infinity.
 - None, because the expression terminates abnormally without a return value.
- Q8.** According to De Morgan's Laws, which condition is logically equivalent to `not (status == "VIP" or points >= 100)`?
- `status != "VIP" or points < 100`
 - `status != "VIP" and points < 100`
 - `not (status == "VIP") or not (points >= 100)`
 - `status == "VIP" and points < 100`
- Q9.** Which string method checks whether all characters in string `s` are digits and `s` has at least one character?
- `s.isnumeric_int()`
 - `s.count()`
 - `s.isdigit()`
 - `s.all_digits()`
- Q10.** What is the minimum number of test cases required to achieve full branch coverage for an `if-elif-elif-else` construct?
- 1 test case
 - 3 test cases
 - 4 test cases (one for `if`, one for each of the two `elif` blocks, and one for `else`)
 - 8 test cases

Section B: Output Tracing, Logic Diagnosis & Error Correction

(25 points)

Answer all 5 questions. Each question carries 5 points. Suggested time: 15 minutes.

Q11. Exact Output Tracing with Nested Decisions & Surcharges (5 points)

Predict the **exact screen output**, including punctuation, spaces, and line breaks:

```
age = 19
is_student = True
base_fare = 40.0
discount = 0.0

if age < 12:
    discount = 0.50
elif age < 25:
    if is_student:
        discount = 0.30
    else:
        discount = 0.15
elif age >= 65:
    discount = 0.40
else:
    discount = 0.0

final_fare = base_fare * (1.0 - discount)
print(f"Base: ${base_fare:.2f} | Discount: {discount * 100:.0f}%")
print(f"Payable: ${final_fare:.2f}")
print(f"Eligible: {discount >= 0.30 and age >= 18}")
```

Output Box for Q11

Q12. Evaluation of Boolean Expressions and Short-Circuiting (5 points)

Write the exact boolean result (True or False) for each expression:

- $10 < 20 \leq 25$: _____
- "py" in "Python".lower() and not (5 > 8): _____
- False or not False and not True: _____
- "cat" > "Cat" and "10" < "2": _____
- len("neu") == 3 or (1 / 0 == 1): _____

Q13. Diagnose and Correct Common Decision Logic Errors (5 points)

For each code scenario, state the underlying defect and provide the corrected Python statement.

Code / Situation	Problem / Logic Flaw	Correction / Valid Code
choice = input("Option: ") if choice == "A" or "B": print("Accepted")		
x = 0.1 + 0.2 if x == 0.3: print("Equal")		
val = input("Enter int: ") if int(val) > 0: (Risk: user enters "abc")		

Q14. String Analysis Methods and Substring Inspection (5 points)

Predict the exact screen output of the following verification routine:

```
code = "NEU-DSFE2026"
is_prefix = code.startswith("NEU")
has_dash = "-" in code
suffix_digits = code[-4:].isdigit()
is_clean = code.isalnum()
print(f"Prefix: {is_prefix}, Dash: {has_dash}")
print(f"Suffix Digits: {suffix_digits}")
print(f"Alphanumeric: {is_clean}")
```

Output Box for Q14**Q15. De Morgan's Laws and Boundary Test Cases (5 points)**

- Apply De Morgan's Law to rewrite the condition without the outer not:
Original: not (temp >= 18 and temp <= 26)
Rewritten: _____
- State the **critical boundary values** that must be tested to thoroughly verify $18 \leq \text{temp} \leq 26$: _____

Section C: Program Design & Logic Specification

(25 points)

Answer all tasks. Suggested time: 15 minutes.

Problem Specification: Air Cargo & Express Courier Pricing Engine

A logistics platform computes parcel delivery cost based on weight w (kg), service speed ("S" for Standard, "E" for Express), and corporate membership status (`is_member`):

- **Input Validation:** Weight must satisfy $0 < w \leq 50.0$; service must be "S" or "E". Otherwise, reject as invalid.
- **Base Tiers:** Tier 1 ($0 < w \leq 2$ kg): flat \$10.00. Tier 2 ($2 < w \leq 10$ kg): \$10.00 + \$3.50/kg for weight over 2.0 kg. Tier 3 ($10 < w \leq 50$ kg): \$38.00 + \$2.00/kg for weight over 10.0 kg.
- **Express Surcharge:** If `service == "E"`, add 40% surcharge on base cost (minimum surcharge \$8.00).
- **Member Discount:** If `is_member` is True and subtotal exceeds \$30.00, deduct a \$5.00 loyalty credit.

Task 1: Identify Decision Criteria, Inputs & Outputs (6 points)

- **Inputs & Types:** _____
- **Validation Criteria:** _____
- **Outputs:** _____

Task 2: Write the Structured Decision Logic in Ordered Steps (7 points)

Formulate sequential pseudo-logic showing the order of validation, tiered branching, surcharges, and discount.

Program-Logic Work Area

Task 3: Hand Trace / Test Case Calculation (6 points)

For $w = 6.0$ kg, `service = "E"`, and `is_member = True`, show step-by-step arithmetic and final payable cost.

Calculation Box for Task 3

Task 4: Explain Two Design Decisions (6 points)

- Why is an `if-elif-else` ladder mandatory rather than consecutive independent `if` statements for tiered pricing?
- Why must input validation be executed **prior** to calculating tiered prices or surcharges?

Section D: Applied Programming in Python

(30 points)

Write complete, syntactically correct Python statements. Apply proper conditional statements (*if*, *elif*, *else*), boolean operators, and input validation. Do **not** define custom functions (*def*) or loops. Suggested time: 18 minutes.

Problem 1: Residential Electricity Billing with Progressive Tiered Tariff (15 points)

Write a Python program that calculates the monthly electricity bill under a progressive tiered tariff structure with an excess surcharge and VAT.

Requirements:

1. Prompt for the customer name `customer_name` (string).
2. Prompt for previous meter reading `prev_reading` and current reading `curr_reading` in kWh (convert to float).
3. **Input Validation:** Both readings must be ≥ 0 , and `curr_reading` $\geq$ `prev_reading`. If invalid, display an error message and do not compute the bill.
4. Compute electricity consumed: `kwh = curr_reading - prev_reading`.
5. Calculate the progressive energy charge: Tier 1 (\$0.12/kWh for first 50 kWh), Tier 2 (\$0.15/kWh for next 50 kWh, 51–100), Tier 3 (\$0.20/kWh for next 100 kWh, 101–200), and Tier 4 (\$0.25/kWh for consumption exceeding 200 kWh).
6. **Excess Surcharge:** If total consumption exceeds 200 kWh, add a flat surcharge of \$10.00.
7. **VAT Calculation:** Compute 8% VAT on the subtotal (energy charge + surcharge).
8. Display formatted receipt: Customer Name, Consumed kWh (1 decimal), Energy Charge, Surcharge, VAT (8%), and Total Bill (all monetary values formatted to 2 decimals with \$).

The output should follow this format when valid:

```
Customer Name: Nguyen Van A
Electricity Consumed: 240.0 kWh
Energy Charge: $43.50
Excess Surcharge: $10.00
VAT (8%): $4.28
Total Bill: $57.78
```

Python Solution for Problem 1

Problem 2: University Course Enrollment & Eligibility Verification (15 points)

Write a Python program that validates student eligibility to register for an advanced course (DSAI2001 - Machine Learning).

Requirements:

1. Prompt for: `student_id` (string), `gpa` (float), `credits_earned` (int), `prereq_status` ("pass"/"fail"), and `warning_flag` ("yes"/"no").
2. **Student ID Format Validation:** ID must be 8 characters, start with "NEU", and last 5 characters must be digits (`student_id[3:].isdigit()`). If invalid, display: "ERROR: Invalid Student ID format." and terminate.
3. **Eligibility Evaluation:** Eligible if and only if `prereq_status.lower() == "pass"`, `warning_flag.lower() == "no"`, `credits_earned ≥ 30`, and `gpa ≥ 2.50`.
4. **Enrollment Classification:** If eligible and `gpa ≥ 3.60` and `credits_earned ≥ 60`: status is "APPROVED (Honors Priority)". If eligible otherwise: status is "APPROVED (Standard)". If ineligible: status is "REJECTED", stating the reason (e.g. missing prerequisite, low GPA, or warning).
5. Display the formatted result card bordered with divider lines ("`=`" * 45).

For example, given NEU12345, 3.75, 64, pass, and no:

```
=====
ENROLLMENT ELIGIBILITY REPORT: DSAI2001
-----
Student ID: NEU12345 | Cumulative GPA: 3.75
Completed Credits: 64
Enrollment Status: APPROVED (Honors Priority)
=====
```

Python Solution for Problem 2

— END OF EXAM PAPER —